

CPD project 1

Part 1: Performance evaluation of a single core

In this project, in groups of 3 students, we will study the effect on the processor performance of the memory hierarchy when accessing large amounts of data. The product of two matrices will be used for this study. Use the Linux Perf tool to collect relevant performance indicators of the program execution.

1. Download the example file from moodle that contains the basic algorithm in C/C++ that multiplies two matrices, i.e. multiplies one line of the first matrix by each column of the second matrix (matrixproduct.cpp). Implement the same algorithm in another programming language (just one), such as JAVA, C#, Fortran, etc, of your choice. (Without using external libraries or imports such as numpy).

Register the processing time for the two languages of implementation, for input matrices from 1024x1024 to 3072x3072 elements with increments in both dimensions of 512. Use -O2 as optimization flag in C++.

2. Implement a version that multiplies an element from the first matrix by the correspondent line of the second matrix, using the 2 programming languages selected in 1.

Register the processing time of the algorithm, in the 2 versions, for input matrices from 1024x1024 to 3072x3072 elements with increments in both dimensions of 512.

Register the processing time from 4096x4096 to 10240x10240 with intervals of 2048 in C/C++ version.

3. Implement a block oriented algorithm that divides the matrices in blocks and uses the same sequence of computation as in 2, using C/C++.

Register the processing time from 4096x4096 to 10240x10240 with intervals of 2048, for different block sizes (e.g. 128, 256, 512).

Outputs:

- Computational complexity of all versions is $2n^3$ Flops, for matrixes of size $n \times n$. Represent in one or more graphs the performance achieved in GFlop/s for all experiments. Note that: $\text{Performance achieved} = 2n^3 / \text{Execution_Time}$.
- Measures relevant processor counters with *perf*.
- Compare and draw conclusions.

Example commands with *perf*:

```
perf stat ls
```

```
perf stat -a ls
```

- To get information related to L1 and L2 cache misses:

```
perf stat -e mem_load_retired.l1_miss,mem_load_retired.l2_miss ./a.out
```

Explore other counters.

CPD project 1

Part 2: Performance evaluation of a multi-core implementation

1. Implement parallel versions of the first and second implementations of the matrix product. Analyze the next two incomplete solutions (for Version 1), and verify their performance (GFlops, speedup and efficiency). Do the same analysis for Version 2.
Note: consider 4 threads and matrices sizes from 1024 to 3072 with increments of 512.

```
#pragma omp parallel for
for (int i=0; i<n; i++)
    for (int j=0; j<n; j++)
    {
        temp = 0
        for (int k=0; k<n; k++)
        {
            temp += a(i,k) * b(k,j)
        }
        C(i,j) = temp
    }
```

```
#pragma omp parallel
for (int i=0; i<n; i++)
    for (int j=0; j<n; j++)
    {
        temp = 0;
        #pragma omp for
        for (int k=0; k<n; k++)
        {
            temp += a(i,k) * b(k,j)
        }
        C(i,j) = temp
    }
```

2. Considering ONLY Version 2 (incomplete below)

```
#pragma omp parallel for
for (int i=0; i<n; i++)
    for (int k=0; k<n; k++)
        for (int j=0; j<n; j++)
            C(i,j)+=A(i,k)*B(k,j) }
```

Execute the program for 4, 8, 12, 16, 20, 24 threads for matrixes of size 8192x8192.

Plot the graph of GFlop/s ($GFLOPS = (2n^3) / (tempo \times 10^9)$) and Speedup.

Explore the following directives:

CPD project 1

#pragma omp for simd

#pragma omp parallel for collapse(2)

Compare and draw conclusions.

To be delivered by: 22/03/2026

Put your code in <https://gitlab.up.pt/> in the group repository. Graphs and analysis also in gitlab.

Presentation: week of 23/03/2026

Perf pointers (documentation and installation):

<https://www.brendangregg.com/linuxperf.html>

<https://oopsmonk.github.io/posts/2022-04-28-perf/>